

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import models
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
import numpy as np
import matplotlib.pyplot as plt
```

```
(x_train,_),(x_test,_) = tf.keras.datasets.mnist.load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>  
11490434/11490434 ————— 0s 0us/step

```
x_train = x_train.astype("float32")/255.0
x_test = x_test.astype("float32")/255.0
```

```
x_train.shape, x_test.shape
```

```
((60000, 28, 28), (10000, 28, 28))
```

```
x_train = np.expand_dims(x_train,-1)
x_test = np.expand_dims(x_test,-1)
x_train.shape, x_test.shape
```

```
((60000, 28, 28, 1), (10000, 28, 28, 1))
```

```
autoencoder = models.Sequential()
autoencoder.add(Conv2D(32,(3,3),activation="relu",padding="same",input_shape=(28,28,1)))
autoencoder.add(MaxPooling2D((2,2),padding="same"))
autoencoder.add(Conv2D(64,(3,3),activation="relu",padding="same"))
autoencoder.add(MaxPooling2D((2,2),padding="same"))
autoencoder.add(Conv2D(64,(3,3),activation="relu",padding="same"))
autoencoder.add(UpSampling2D((2,2)))
autoencoder.add(Conv2D(32,(3,3),activation="relu",padding="same"))
autoencoder.add(UpSampling2D((2,2)))
autoencoder.add(Conv2D(1,(3,3),activation="sigmoid",padding="same"))
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base\_conv.py:113: UserWarning: Do not pass an `input\_shape` argument to `Conv2D` layers. It will be ignored. In the future, this will result in an error.  
super().\_\_init\_\_(activity\_regularizer=activity\_regularizer, \*\*kwargs)

```
autoencoder.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

```
history = autoencoder.fit(x_train,x_train,batch_size=256,epochs=50,shuffle=True,validation_data=(x_test,x_test))
```

```

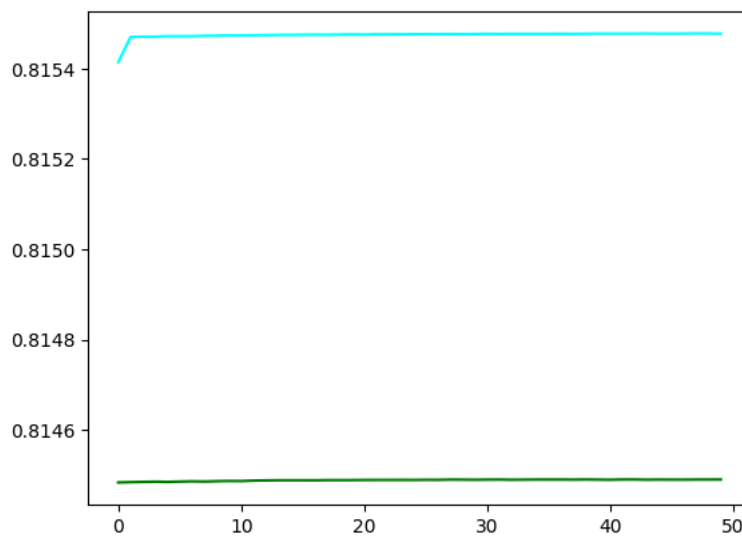
Epoch 39/50
235/235 ————— 3s 13ms/step - accuracy: 0.8152 - loss: 0.0625 - val_accuracy: 0.8145 - val_loss: 0.0622
Epoch 40/50
235/235 ————— 3s 12ms/step - accuracy: 0.8155 - loss: 0.0624 - val_accuracy: 0.8145 - val_loss: 0.0623
Epoch 41/50
235/235 ————— 3s 12ms/step - accuracy: 0.8152 - loss: 0.0626 - val_accuracy: 0.8145 - val_loss: 0.0622
Epoch 42/50
235/235 ————— 3s 12ms/step - accuracy: 0.8158 - loss: 0.0624 - val_accuracy: 0.8145 - val_loss: 0.0622
Epoch 43/50
235/235 ————— 3s 13ms/step - accuracy: 0.8152 - loss: 0.0624 - val_accuracy: 0.8145 - val_loss: 0.0622
Epoch 44/50
235/235 ————— 3s 12ms/step - accuracy: 0.8156 - loss: 0.0625 - val_accuracy: 0.8145 - val_loss: 0.0621
Epoch 45/50
235/235 ————— 3s 12ms/step - accuracy: 0.8154 - loss: 0.0624 - val_accuracy: 0.8145 - val_loss: 0.0620
Epoch 46/50
235/235 ————— 3s 12ms/step - accuracy: 0.8156 - loss: 0.0623 - val_accuracy: 0.8145 - val_loss: 0.0620
Epoch 47/50
235/235 ————— 3s 13ms/step - accuracy: 0.8155 - loss: 0.0624 - val_accuracy: 0.8145 - val_loss: 0.0620
Epoch 48/50
235/235 ————— 3s 12ms/step - accuracy: 0.8152 - loss: 0.0624 - val_accuracy: 0.8145 - val_loss: 0.0620
Epoch 49/50
235/235 ————— 3s 12ms/step - accuracy: 0.8159 - loss: 0.0622 - val_accuracy: 0.8145 - val_loss: 0.0620
Epoch 50/50
235/235 ————— 3s 12ms/step - accuracy: 0.8156 - loss: 0.0623 - val_accuracy: 0.8145 - val_loss: 0.0619

```

```

plt.plot(history.history["accuracy"], label="Train Accuracy", color="cyan")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy", color="green")
plt.show()

```



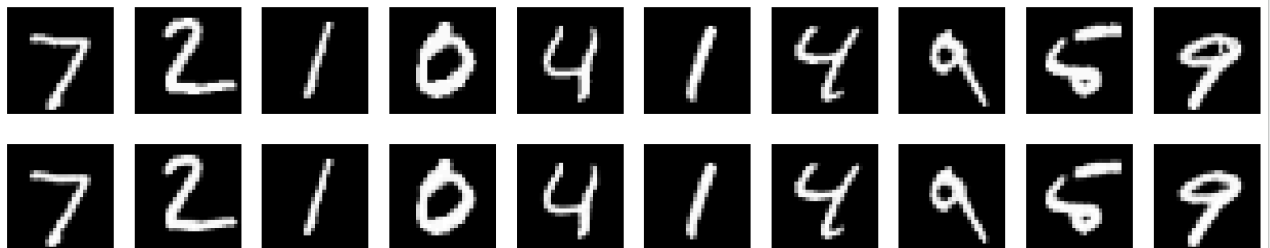
```

decoded_imgs = autoencoder.predict(x_test)
n = 10
plt.figure(figsize=(20,4))
for i in range(n):
    ax = plt.subplot(2,n,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap="gray")
    ax.axis("off")

    ax = plt.subplot(2,n,i+1+n)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap="gray")
    ax.axis("off")
plt.show()

```

313/313 ————— 1s 3ms/step



Start coding or [generate](#) with AI.

